SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering

Somaiya TRUST

Modelling Concpets, Characteristics, Views and View Points, Inconsistencies

Group A23

Nilesh Pany

Yuvraj Patil

Hyder Presswala

Introduction

- Software systems are often large and complex, making them difficult to understand.
- Modelling provides a simplified representation of these systems.
- It helps in visualizing, designing, and communicating the structure and behavior of a system.
- Serves as a bridge between requirements, design, and implementation.
- Enables clear communication among architects, developers, and stakeholders.
- Plays a vital role in Software Architecture and Design Thinking, supporting better understanding and decision-making.

What is Modelling?

- Modelling is the process of creating a simplified representation of a real-world system.
- It focuses on essential features while ignoring unnecessary details.
- Helps in analyzing, designing, and understanding software systems.
- Provides a visual view of how components interact and behave.
- Used throughout the software development life cycle (SDLC) — from analysis to implementation.
- Helps bridge the gap between conceptual ideas and technical execution.

Why Do We Use Modelling?

- Helps in reducing complexity of large and detailed systems.
- Improves communication between team members and stakeholders.
- Supports better decision-making during design and development.
- Enables early detection of errors or design flaws before implementation.
- Acts as a blueprint for system development and documentation.
- Makes systems easier to maintain, modify, and scale in the future.

Modelling Concepts

- Modelling is the foundation of software architecture and design thinking.
- It represents how a system is structured, behaves, and interacts with other components.
- Models can describe different aspects of a system — structure, data, behavior, and control flow.
- They help convert real-world problems into abstract, understandable forms.
- Modelling provides a shared vision among developers, designers, and stakeholders.

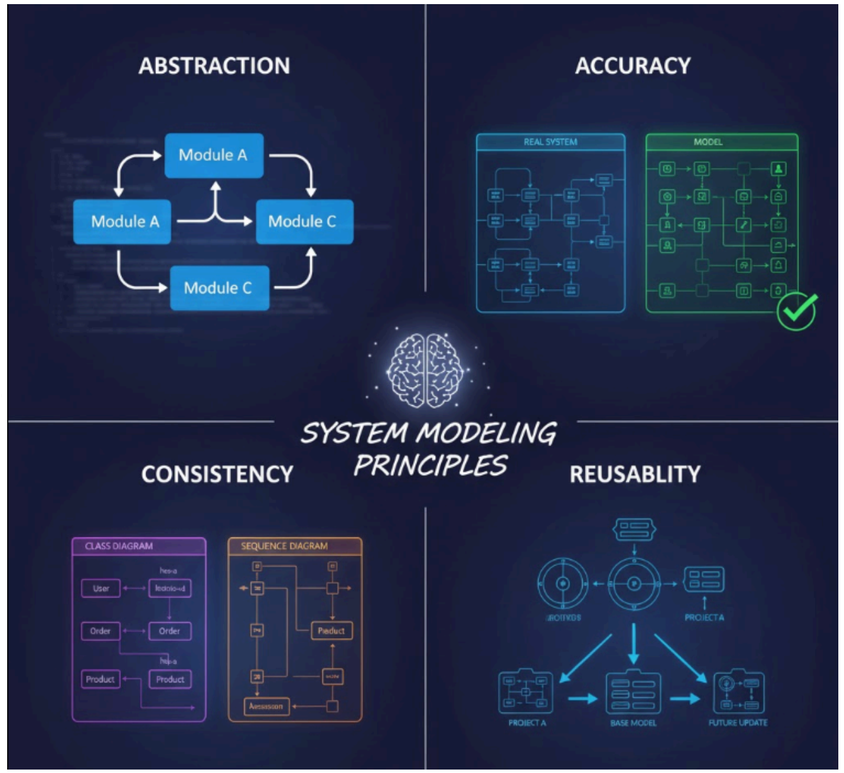
Characteristics of a Good Model

Abstraction

- A good model captures only the essential details, ignoring unnecessary complexity.
- It focuses on what's important for understanding the system's behavior and structure.
- Helps stakeholders focus on "what the system does" instead of "how it's implemented."
- Example: Showing modules and their interactions instead of low-level code logic.

Accuracy

- The model should represent the real system as closely as possible.
- It must correctly show relationships, dependencies, and behaviors of components.
- Inaccurate models can lead to wrong design decisions or implementation errors.
- Example: A use case diagram that accurately shows all user interactions.



Characteristics of a Good Model

Consistency

- The model must be logically and structurally consistent.
- There should be no conflicts between different models or views of the same system.
- Consistency ensures different parts of the model align with each other.
- Example: The class diagram and sequence diagram should not contradict each other.

Reusability

- A well-structured model can be reused for similar projects or as a base for future updates.
- Encourages standardization and saves design time.
- Helps in maintaining a consistent architecture across projects.
- Example: Reusing data flow models for similar systems.

What are Views?

Definition

- A view is a purposeful representation of a software architecture from a specific perspective to address particular stakeholder concerns.

Why We Use Views

- Software systems are multidimensional & complex
- No single diagram can explain everything
- Different stakeholders → Different priorities

Key Ideas

- A view focuses on what matters for a specific audience
- Helps visualize, specify, construct, and document architecture
- Prevents information overload

Stakeholder	Focus	Sample View
Developers	Code design	Class diagrams
Testers	Behavior & flows	Sequence diagrams
Business Owners	Functionality	Use-case diagrams
DevOps	Infrastructure	Deployment diagrams

Types of Views

🔹 Logical View

Represents the functional structure of the system, showing key components, classes, and their relationships.

Focus: System functionality, object model
Audience: Software engineers, analysts
Artifacts: Class diagrams, data models
Example: Customer, Payment, Order classes in an e-commerce system

🔹 Process View

Describes the runtime behavior of the system, including processes, threads, and communication between them.

Focus: Runtime behavior, concurrency, threads, communication
Audience: Performance engineers, DevOps
Artifacts: Activity, Sequence diagrams
Example: Asynchronous payment processing flow

🔹 Development / Implementation View

Shows how the system is organized in terms of modules, packages and source code structure.

Focus: Code structure, build organization, components
Audience: Developers
Artifacts: Package diagrams, Layer diagrams
Example: UI Layer → Business Layer → Database Layer

🔹 Physical / Deployment View

Represents how the software is deployed on hardware infrastructure such servers, networks, and nodes.

Focus: Hardware layout, network topology, nodes
Audience: Infra team, Cloud architects
Artifacts: Deployment diagrams
Example: Web server + App server + DB server + Load balancer

Why Views are Critical

✅ Why Views are Critical

Manage complexity via separation of concerns

Modern systems are large and multi-layered; views break the system into understandable sections so each team can focus on the part relevant to them (e.g., design, runtime, deployment) without getting overwhelmed.

Reduce misunderstanding across teams

Different people think differently — business sees features, developers see modules, DevOps sees servers. Views ensure everyone's interpretation of the system is aligned and consistent.

Make architecture reviewable & traceable

Views allow stakeholders to validate and verify design decisions. They create a trail linking requirements → design → implementation → deployment, ensuring accountability and clarity.

Support architectural decision making

With views, architects can evaluate trade-offs (performance, scalability, security, cost) more accurately, and choose the best approach backed by visual proof and structured representation.

Enable reusability & maintainability

By clearly defining modules, interfaces, and interactions, views help teams reuse components easily and make future maintenance efficient instead of messy or risky.

✅ Primary Benefits

✅ Clear communication between stakeholders

Views act as a shared language, helping non-technical and technical teams communicate the same system vision effectively.

✅ Better planning & design before coding

With well-defined views, teams can foresee challenges early, plan architecture safely, and avoid expensive redesigns later in the development cycle.

✅ Helps identify bottlenecks & risks early

Views highlight potential performance bottlenecks (e.g., overloaded servers), security risks (e.g access points), and logical flaws before the system goes into implementation.

✅ Improves system quality (performance, reliability, security)

By examining the system from different angles — functional, structural, behavioral, and deployment — views ensure the final solution is robust, secure, scalable, and high-performing.

Real-World Example:

UPI Payment System:

- Logical view → Payment authorization process
- Process view → Real-time transaction routing
- Deployment view → Bank servers, NPCI cloud, payment switch
- Scenario view → UPI flow: Initiate → Validate → Debit → Credit
  A single diagram cannot show this — **multiple views are mandatory.**

What are Viewpoints?

What are Viewpoints?

A **viewpoint** is a **framework, method, and set of guidelines** that define:

- What a view must contain
- Which modeling language to use
- What concerns it addresses
- How to interpret it

Why They Matter

- Provide **standardization**
- Ensure **consistency across teams**
- Guarantee views are **complete & verified**
- Align model with **stakeholder expectations**

ISO 42010 mandates defining viewpoints.

View	Viewpoint
Actual representation	Rules to create the representation
What you see	How you draw it
Specific diagram	Template & method

Types of Viewpoints & Examples

Common Viewpoint Categories

Viewpoint	Purpose	Target Stakeholder	Output
Functional	Describes what system does	Users, Analysts	Use-case / Function models
Information / Structural	Component structure & data	Developers	UML Class, Component diagrams
Behavioral	Interaction & runtime dynamics	Testers, Designers	Sequence & Activity diagrams
Deployment	Environment & runtime placement	Cloud/Infra team	Deployment diagrams
Security	Security controls & risks	Security engineers	Threat models, Access control view

Example Mapping

System	Viewpoint	Example
E-commerce security	Security viewpoint	OTP & Encryption flow
Bank database	Information viewpoint	ER diagram
Hospital software	Functional viewpoint	Patient admission use-case

Inconsistencies in Models

Inconsistency in models occurs when different parts of a system model contradict each other, leading to confusion and potential errors in software design or implementation.

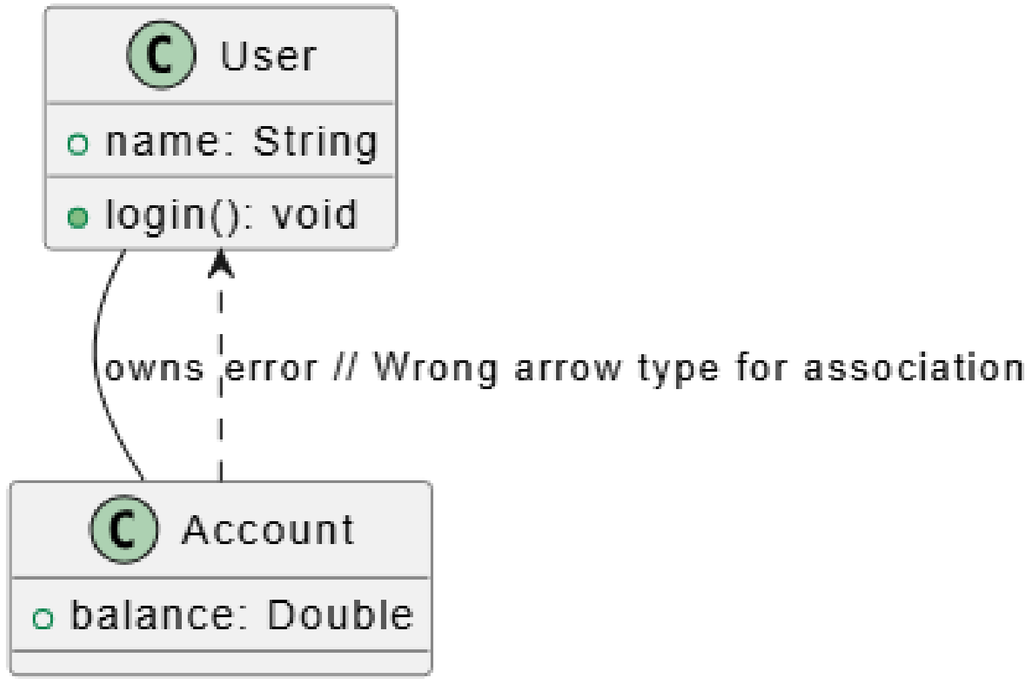
Types of Inconsistencies

1. Syntactic Inconsistency
2. Semantic Inconsistency
3. Viewpoint Inconsistency

1. Syntactic Inconsistency

Syntactic inconsistency happens when the model violates the rules or syntax of the modeling language.

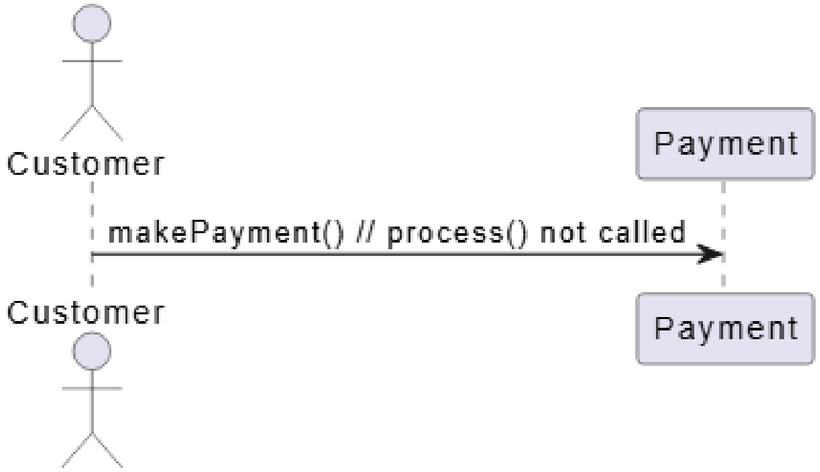
Example:-

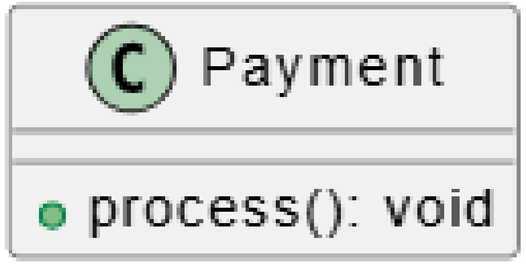


2.Semantic Inconsistency

Semantic inconsistency occurs when the meaning or logic of model elements contradict each other

Example:-

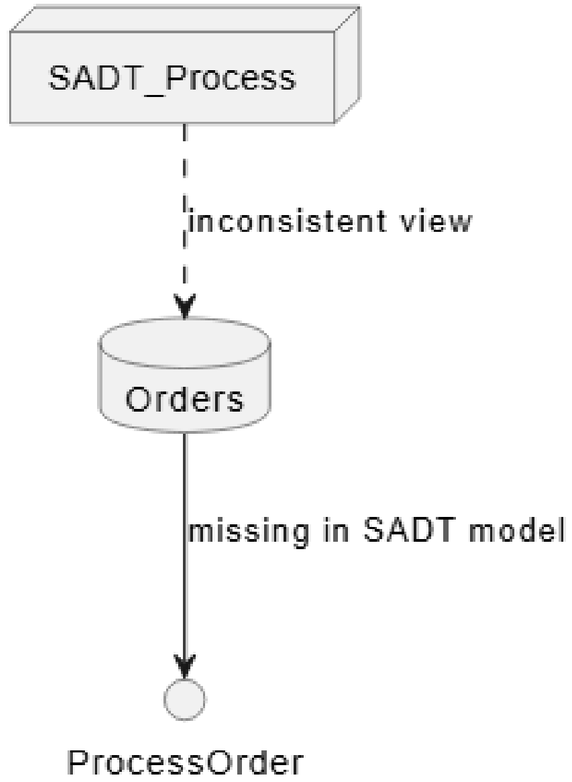




3.Viewpoint Inconsistency

Viewpoint inconsistency is when different views or perspectives of the system present conflicting information.

Example:-



SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering

Somaiya TRUST

Q & A

SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering

Somaiya TRUST

Thank You!

SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering

link

Somaiya TRUST

https://drive.google.com/file/d/1Xj0E3xZGfzHrYV5yuzs8eXhZRQRES-rT/view?usp=sharing